Báo cáo Tuần

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Môn: Lập trình Đa nền tảng

Chủ đề 03: Xử lý tác vụ nặng với Isolate (Hàm compute) để giữ mượt giao diện

SVTH:
1. [Họ và tên Thành viên 1] – MSSV: [MSSV 1]
2. [Họ và tên Thành viên 2] – MSSV: [MSSV 2]

Đà Nẵng, 2026

Nội dung báo cáo

1. **Phân công công việc (WBS) & Kiến trúc thư mục dự án**
2. **Phần 1: Cơ chế Main Thread, Event Loop & Hiện tượng UI Jank/Freeze**
   - Ngưỡng 16.6ms (60 FPS) và bài toán giữ nhịp giao diện với `AnimationMonitor`.
   - Tại sao `async/await` không giải quyết được tác vụ nặng ngốn CPU (CPU-bound)?
3. **Phần 2: Kiến trúc Dart Isolate & Giải pháp hàm `compute()`**
   - Cơ chế Cô lập bộ nhớ (Isolated Heap) và Truyền thông điệp (Message Passing).
   - Chu trình 4 bước của `compute()` gắn liền với cách viết Top-level Function trong code.
4. **Phần 3: Đối chiếu Thực thi Code – Main Thread vs `compute()`**
   - So sánh trực tiếp `executeOnMainThread` và `executeWithCompute`.
   - Quản lý trạng thái bất đồng bộ an toàn (`mounted`) và xử lý ngoại lệ chặt chẽ.
5. **Phần 4: Tổng kết & Tài liệu tham khảo**

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG PHỤ TRÁCH	ĐÓNG GÓP
1	[Tên Thành viên 1]	- Nghiên cứu lý thuyết Main Thread, Event Loop, Isolate. - Thiết kế `task_result_model.dart` và `task_execution_exception.dart`. - Xây dựng các hàm Top-level tính toán nặng và lớp `HeavyTaskService`.	50%
2	[Tên Thành viên 2]	- Nghiên cứu luồng dữ liệu (Data Flow) của hàm `compute()`. - Xây dựng bộ đo `animation_monitor.dart` và thẻ `result_card.dart`. - Lập trình màn hình `isolate_demo_screen.dart` và tài liệu `README.md`.	50%

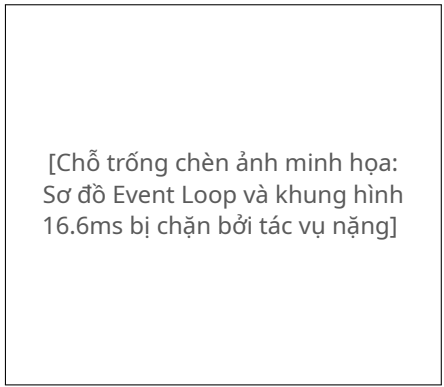
1.1. Cơ chế Main Thread, Event Loop & Ngưỡng 16.6ms

Luồng thực thi mặc định của Flutter:

- Mọi ứng dụng Flutter khởi chạy trên một luồng chính duy nhất gọi là **Main Isolate (UI Thread)**.
- Bên trong Main Thread là **Event Loop** xử lý tuần tự: Nhận thao tác chạm $\rightarrow$ Chạy logic Dart $\rightarrow$ Dựng hình (Render UI).

Quy tắc 16.6ms và hiện tượng UI Freeze:

- Để đạt độ mượt **60 FPS**, mỗi khung hình phải được vẽ xong trong tối đa $\frac{1000 \text{ ms}}{60} \approx$ **16.6 ms**.
- Nếu một hàm tính toán chiếm dụng Main Thread liên tục từ 2–4 giây, Event Loop bị kẹt cứng $\rightarrow$ **Giao diện đứng hình (UI Freeze)**.



1.2. Minh họa trong Code: Bộ kiểm tra nhịp vẽ UI (`AnimationMonitor`)

Mục đích thiết kế trong dự án:

- Để quan sát trực quan việc Event Loop có kịp vẽ khung hình mỗi 16.6ms hay không, nhóm xây dựng widget `AnimationMonitor`.
- Kết hợp `CircularProgressIndicator` và `LinearProgressIndicator` chuyển động liên tục không ngừng nghỉ trên UI Thread.
- **Dấu hiệu nhận biết:** Ngay khi Main Thread bị nghẽn, cả vòng xoay tròn lẫn thanh chạy ngang sẽ lập tức đứng khựng lại.

Trích đoạn `animation_monitor.dart`: `class`

```
AnimationMonitor extends StatelessWidget {
  const AnimationMonitor({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      // ...
      child: Row(
        children: [
          SizedBox(
            width: 48, height: 48,
            child:
CircularProgressIndicator(),
          ),
          Expanded(
            child:
LinearProgressIndicator(),
          ),
        ],
      ),
    );
  }
}
```

1.3. Tại sao async/await không chống đơ được cho tác vụ nặng?

Phân biệt I/O-bound và CPU-bound:

- **Tác vụ I/O-bound (Gọi API, đọc Database):** Thời gian chủ yếu là chờ phản hồi. Dùng `async/await` giúp Main Thread rảnh tay vẽ UI trong lúc chờ.
- **Tác vụ CPU-bound (Vòng lặp tỷ lần, Parse JSON lớn):** Bắt nhân CPU phải tính toán liên tục 100% công suất, không có khoảng nghỉ.

Bản chất trong dự án:

- Dù hàm `executeOnMainThread` có khai báo `async/await`, nhưng khi bước vào vòng lặp 1,5 tỷ lần trên cùng Main Thread, giao diện vẫn đơ hoàn toàn.

Minh chứng trong `heavy_task_service.dart`:

```
Future<TaskResultModel>
executeOnMainThread(...) async {
  final stopwatch = Stopwatch()..start();
  // Nhường 50ms để UI kịp hiện chữ "Đang
xử lý..."
  await Future<void>.delayed(
    const Duration(milliseconds: 50),
  );
  // Ngay sau dòng này, Main Thread bị
khóa chặt!
  final result =
_runHeavyLoopTask(1500000000);
  // ...
                        }
```

[Chỗ trống chèn ảnh minh họa:
So sánh I/O-bound (rảnh CPU) vs CPU-bound (nghẽn CPU)]

2.1. Kiến trúc Dart Isolate & Sự khác biệt với Thread truyền thống

Nguyên lý Cô lập bộ nhớ (Isolated Memory):

- Để chạy song song trên nhân CPU khác mà không làm đơ UI, Dart sử dụng **Isolate**.
- Mỗi Isolate có **vùng nhớ Heap riêng biệt** và **Event Loop riêng**, tuyệt đối không dùng chung biến.

Tiêu chí đối chiếu	Thread truyền thống (C++ / Java)	Dart Isolate (Flutter)
Vùng nhớ (Memory)	Chia sẻ bộ nhớ chung (*Shared Memory*).	Cô lập hoàn toàn (*Isolated Heap*).
Rủi ro xung đột	Dễ xảy ra *Race Condition*, cần khóa *Lock/Mutex*.	An toàn tuyệt đối, không cần cơ chế khóa phức tạp.
Cơ chế giao tiếp	Đọc/ghi trực tiếp vào cùng ô nhớ.	Truyền thông điệp (*Message Passing*) qua `Port`.
Dọn rác bộ nhớ (GC)	Tạm dừng toàn bộ ứng dụng (*Stop-the-world*).	Dọn rác độc lập từng Isolate, giữ mượt UI Thread.

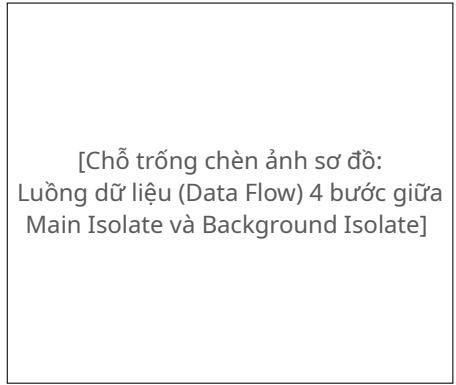
2.2. Giải pháp hàm `compute()` & Chu trình 4 bước thực thi

Tại sao sử dụng `compute(callback, message)`?

- Thay vì tự khởi tạo `Isolate.spawn` và quản lý `SendPort/ReceivePort` thủ công phức tạp, Flutter đóng gói toàn bộ quy trình vào hàm `compute()`.

Chu trình 4 bước diễn ra ngầm:

1. **Spawn:** Tạo mới một *Background Isolate*.
2. **Copy:** Sao chép hàm `callback` và tham số `message` sang vùng nhớ của Isolate mới.
3. **Execute:** Chạy hàm nặng trên nhân CPU riêng; *Main Isolate* vẫn tự do vẽ UI 60 FPS.
4. **Return & Kill:** Trả kết quả về luồng chính và tự động hủy Isolate ngầm để giải phóng RAM.



2.3. Áp dụng Lý thuyết vào Code: Quy tắc viết hàm cho `compute()`

2 Quy tắc bắt buộc từ kiến trúc Isolate:

- **Quy tắc 1 (Top-level / Static):** Vì Isolate không chia sẻ bộ nhớ, hàm truyền vào `compute` phải đặt **ngoài mọi class** (Top-level) hoặc là `static` để không kéo theo ngữ cảnh của Widget/Class sang luồng khác.
- **Quy tắc 2 (Đúng 01 tham số):** Hàm chỉ nhận duy nhất 1 đầu vào (`int iterations` hoặc `int itemCount`).

Mã nguồn trong `heavy_task_service.dart`: `// Đặt độc lập ở đầu file (ngoài class HeavyTaskService)`

```
int _runHeavyLoopTask(int iterations) {
  if (iterations <= 0) throw ArgumentError('...');
  int sum = 0;
  for (int i = 0; i < iterations; i++) {
    sum += (i % 7) * (i % 3);
  }
  return sum;
}

int _runHeavyJsonParseTask(int itemCount) {
  final rawList = List.generate(itemCount, (i) =>
{
    'id': i, 'name': 'Item #$i', 'score': i * 1.5,
  });
  final jsonString = jsonEncode(rawList);
  final decoded = jsonDecode(jsonString) as List;
  return decoded.length;
}
```

3.1. Tổ chức Code: Cấu trúc thư mục, Model & Xử lý Ngoại lệ

Kiến trúc tách lớp rõ ràng:

- `core/exceptions/`: Đóng gói lỗi thực thi bằng `TaskExecutionException`.
- `models/`: Dùng `enum` định danh rõ bài toán (`TaskType`) và chế độ chạy (`ExecutionMode`), trả về đối tượng `TaskResultModel` đồng nhất.

[Chỗ trống chèn ảnh chụp cây thư mục `lib/` trong IDE]

Trích đoạn `task_result_model.dart` & `exception`:

```
enum TaskType {
  heavyLoop('Vòng lặp lớn (1.5 tỷ lần)'),
  jsonParse('Parse danh sách JSON (50.000 mục)');
  final String label; const TaskType(this.label);
}

class TaskResultModel {
  final TaskType taskType;
  final ExecutionMode mode;
  final String outputSummary;
  final int durationMs;
  // ...
}

class TaskExecutionException implements Exception
{
  final String message;
  final Object? originalError;
  const TaskExecutionException(this.message,
[this.originalError]);
}
```

3.2. Trọng tâm Code: Đối chiếu `Main Thread` và `compute()`

Nút 1: `executeOnMainThread` (Đơ UI)

```
final stopwatch = Stopwatch()..start();
try {
  await Future<void>.delayed(
    const Duration(milliseconds: 50));
  String summary = '';
  switch (taskType) {
    case TaskType.heavyLoop:
      final res =
_runHeavyLoopTask(1500000000);
      summary = 'Tổng tính được: $res';
    case TaskType.jsonParse:
      final cnt =
_runHeavyJsonParseTask(50000);
      summary = 'Đã parse thành công
$cnt...';
  }
  stopwatch.stop();
  return TaskResultModel(...);
} catch (e) {
  throw TaskExecutionException('...', e);
}
```

Nút 2: `executeWithCompute` (Mượt UI)

```
final stopwatch = Stopwatch()..start();
try {
  String summary = '';
  switch (taskType) {
    case TaskType.heavyLoop:
      final res = await compute(
        _runHeavyLoopTask, 1500000000);
      summary = 'Tổng tính được: $res';
    case TaskType.jsonParse:
      final cnt = await compute(
        _runHeavyJsonParseTask, 50000);
      summary = 'Đã parse thành công
$cnt...';
  }
  stopwatch.stop();
  return TaskResultModel(...);
} catch (e) {
  throw TaskExecutionException('...', e);
}
```

Điểm khác biệt cốt lõi: Thay lời gọi trực tiếp bằng `await compute(callback, arg)` để giải phóng hoàn toàn Main Thread.

3.3. Giải thích Code: Điều phối State tại `IsolateDemoScreen`

Hàm xử lý sự kiện `_handleRun(ExecutionMode mode)`:

```
Future<void> _handleRun(ExecutionMode mode) async
{
  if (_isRunning) return; // Chống bấm trùng lặp
  setState(() { _isRunning = true; _errorMessage =
null; });
  try {
    final TaskResultModel result;
    if (mode == ExecutionMode.mainThread) {
      result = await
_taskService.executeOnMainThread(_selectedTask);
    } else {
      result = await
_taskService.executeWithCompute(_selectedTask);
    }
    if (!mounted) return;
    setState(() { _lastResult = result; });
  } on TaskExecutionException catch (e) {
    if (mounted) setState(() { _errorMessage =
e.toString(); });
  } finally {
    if (mounted) setState(() { _isRunning = false;
});
  }
}
```

Các kỹ thuật đảm bảo chất lượng Code:

- **Chặn thao tác thừa (`_isRunning`):** Vô hiệu hóa 2 nút bấm và thanh chọn `SegmentedButton` khi tác vụ đang chạy.
- **Kiểm tra `mounted`:** Vì tác vụ chạy tốn vài giây, việc kiểm tra `if (!mounted) return;` trước khi gọi `setState` giúp tránh lỗi cập nhật UI khi màn hình đã bị hủy.
- **Bắt ngoại lệ phân tầng:** Tách riêng on `TaskExecutionException` và khối `finally` để luôn tắt trạng thái loading.

4.1. Sơ lược Ứng dụng Demo & Hiển thị Kết quả (`ResultCard`)

Cơ chế hiển thị của `ResultCard`:

- Khi `isRunning == true`: Hiển thị biểu tượng đồng hồ cát *"Đang xử lý tác vụ nặng..."*.
- Khi hoàn tất: Tự động phân biệt màu sắc và icon cảnh báo đỏ (Main Thread) hoặc tích xanh (`compute`), kèm thời gian đo bằng `Stopwatch` (`durationMs`).

Kịch bản Live Demo trực tiếp (5 phút):

1. Chọn bài toán trên `SegmentedButton`.
2. Bấm **Nút 1 (Đỏ)** → Quan sát `AnimationMonitor` đứng hình.
3. Bấm **Nút 2 (Xanh)** → Quan sát `AnimationMonitor` quay mượt mà.

[Chỗ trống chèn ảnh chụp màn hình: Giao diện App khi chạy bằng Main Thread và khi chạy bằng hàm `compute()`]

4.2. Tổng kết & Tài liệu tham khảo

Kết luận rút ra từ đề tài:

- **Khi nào cần dùng `compute()`?** Khi xử lý các tác vụ **CPU-bound** làm thời gian xử lý vượt quá ngưỡng **16.6ms** (như giải mã JSON lớn, thuật toán lặp lớn, xử lý ảnh/mã hóa).
- **Khi nào KHÔNG nên lạm dụng `compute()`?** Với các tác vụ tính toán nhỏ (chỉ vài mili-giây), chi phí khởi tạo Isolate và sao chép dữ liệu (*overhead*) sẽ lớn hơn thời gian chạy trực tiếp.

Tài liệu tham khảo:

1. Flutter Documentation, *Concurrency and isolates in Flutter*, `https://docs.flutter.dev/perf/isolates`.
2. Flutter API Reference, `compute` *function – foundation library*, `https://api.flutter.dev/flutter/foundation/compute.html`.
3. Dart Language Guide, *Concurrency in Dart (Event Loop & Isolates)*, `https://dart.dev/language/concurrency`.
4. Flutter Performance, *UI Jank and Rendering Performance (60fps/120fps)*, `https://docs.flutter.dev/perf/rendering-performance`.